

# LogiFlow — Week 3 Progress Report

**Candidate:** Aryan Ajmera

**Reference Number:** NFGT/HR/INT/2026/160

**Week:** 3 of 8 (Backend Engineering & REST APIs — Weeks 1–4)

**Date:** August 7, 2026

## 1. Tasks Completed

### Task W3.1 — Customer & Order Management APIs

- Implemented full CRUD for Customer: create, list (searchable by name/email, filterable, paginated), get by id, update, and soft delete (accountStatus set to 'Inactive')
- Restricted create/delete to Administrator and Logistics Coordinator; update and GET left open to all authenticated roles, per spec
- Implemented Order creation with per-item stock validation against Inventory before anything is saved, totalAmount calculation, and stock reservation via deduction on save
- Implemented the Order status machine (Draft → Confirmed → Processing → Dispatched → Delivered → Cancelled) with forward-only transitions, a dedicated cancellation path that returns reserved stock to Inventory, and a guard preventing cancellation of an already Delivered/Cancelled order
- Logged an ActivityLog entry on every order status change
- Tested all endpoints in Postman including insufficient-stock, backward-transition, and cancel-after-delivery edge cases

### Task W3.2 — Shipment Lifecycle Management APIs

- Implemented shipment creation from a Confirmed order only, auto-generating trackingId via Node's built-in crypto.randomUUID() and copying originWarehouseId from the linked order
- Implemented the Shipment status machine (Created → Assigned → In-Transit → Out-for-Delivery → Delivered → Failed) with forward-only transitions and a guard preventing a Delivered shipment from later being marked Failed
- Implemented auto-sync: marking a shipment Delivered updates the linked Order's status to Delivered as well
- Implemented a Failed-status Notification trigger, resolving a Logistics Coordinator user via User.findOne() and creating a Notification document addressed to them
- Implemented POST /:id/tracking to append timestamped location/description events to the shipment's trackingHistory array
- Tested the full lifecycle in Postman: creation-from-unconfirmed-order rejection, forward progression, tracking event append, Delivered → Order sync, and Failed → Notification delivery

### Task W3.3 — Week 3 Integration & Review

- Ran the full Customer → Order → Confirm → Shipment → Delivered chain end-to-end, confirming stock deduction, order-status auto-sync, and ActivityLog entries at every step
- Re-verified order cancellation and stock return after later changes to the Inventory/Order code
- Confirmed ActivityLog entries exist for both Order and Shipment status changes with correct entity values

- Committed all work to the develop branch

## 2. Blockers Faced & Resolutions

- **Copy-paste reference mismatches across new controllers:** The Customer model was imported under the name Warehouse (breaking every function but create), a Customer null-check referenced a stray deletedWarehouse variable, and a route file imported updateShipment while the controller exported it under a different name, crashing the server. Resolved by tracing each error to its mismatched identifier and standardising names per file.
- **Async loop misuse in order creation:** An early draft used for...in instead of for...of, tried calling another route handler as a data-fetching function, and sent a response inside a per-item loop, causing multiple responses for one request. Resolved by restructuring into three sequential passes — validate stock, calculate totalAmount, reserve stock — then create the Order and respond once.
- **Status-machine edge cases beyond a simple forward-only check:** A plain 'index must increase' rule allowed cancelling an already Delivered order and marking an already Delivered shipment as Failed. Resolved with explicit carve-out guards for cancellation and failure alongside the general forward-only comparison.
- **Silent no-op on missing Notification recipient:** The Failed-shipment Notification depends on a Logistics Coordinator user existing; with none registered, it failed silently — the same failure pattern as Week 2's managerId issue. Resolved by registering a dedicated Logistics Coordinator test user before verifying this path.
- **Mongoose subdocument array mutation:** An early tracking-event attempt called .push() on the shipment document itself instead of its trackingHistory array, and omitted await on the findById lookup. Resolved by pushing a single {location, description} object into trackingHistory after awaiting the lookup.

## 3. What Was Learned

- Why for...of is required (not for...in) when looping over arrays of objects with async operations inside the loop
- Structuring multi-step business logic as sequential passes — validate, then compute, then mutate — rather than combining everything into one loop, to avoid premature or duplicate responses
- Controller functions are Express route handlers, not reusable helper functions — cross-module logic must query the relevant model directly rather than calling another controller's function
- Designing status machines with a general forward-only rule plus explicit carve-outs for terminal or branching states (cancellation, failure) that don't fit a single linear progression
- Recognising the recurring 'silent no-op on missing reference' failure pattern early, and testing for it proactively rather than after the fact
- Keeping route handler imports and controller export names in sync, and mounting every new route file into server.js immediately after creating it

## 4. Plan for Next Week (Week 4)

- Task W4.1: Vehicle, Driver & Dispatch Planning APIs, including availability validation and state changes on assignment/completion
- Task W4.2: Analytics, Activity Log, Notification, and Support Ticket APIs
- Task W4.3: Week 4 backend integration review, full 5-role permission simulation, and comprehensive API documentation